Rapport de Travaux Pratiques

MongoDB – Prise en main et CRUD

Mini-TP 3, 4, 5 et 6

Étudiant : Asma Daoui[0.3em] **Cours :** NoSQL Security – 2ème année Cybersécurité

Date : 5 avril 2026

Résumé

Ce rapport documente l'ensemble des travaux pratiques réalisés sur MongoDB. Les Mini-TP 3 à 6 couvrent : le déploiement d'un conteneur Docker MongoDB, la création d'une base de données `noseclab` avec une collection `security_logs`, l'insertion d'un jeu de données réaliste de logs de sécurité, la maîtrise des opérations CRUD (Create, Read, Update, Delete), et enfin des scénarios de requêtes typiques d'un analyste SOC (détection de brute-force, activités étrangères, événements critiques).

Table des matières

1 Mini-TP 3 – Démarrage Docker & MongoDB **3**
1.1 Objectif . . . 3
1.2 Commandes exécutées . . . 3
1.3 Bilan du Mini-TP 3 . . . 4

2 Mini-TP 4 – Création de la base et insertion du dataset **5**
2.1 Objectif . . . 5
2.2 Commandes exécutées . . . 5
2.2.1 Insertion des 12 logs de sécurité . . . 7
2.3 Vérifications . . . 8
2.4 Bilan du Mini-TP 4 . . . 9

3 Mini-TP 5 – Exercices CRUD guidés **10**
3.1 Objectif . . . 10
3.2 Exercice A – Create (Ajouter un document) . . . 10
3.3 Exercice B – Read (Lire et filtrer) . . . 11
3.4 Exercice C – Update (Modifier des documents) . . . 11
3.5 Exercice D – Delete (Supprimer un document) . . . 12
3.6 Commandes dangereuses (à ne JAMAIS exécuter en production) . . . 13
3.7 Bilan du Mini-TP 5 . . . 13

4 Mini-TP 6 – Scénarios de requêtes cybersécurité **14**
4.1 Objectif . . . 14
4.2 Scénario 1 – Détection de brute-force . . . 14
4.3 Scénario 2 – Activité depuis l'étranger . . . 15
4.4 Scénario 3 – Les 3 derniers événements critiques . . . 15
4.5 Scénario 4 – Nombre d'événements par type d'action (Aggregation) . . . 16
4.6 Scénario 5 – Exercice libre . . . 16
4.7 Bilan du Mini-TP 6 . . . 17

5 Synthèse et commandes de nettoyage **18**
5.1 Récapitulatif des commandes essentielles . . . 18
5.2 Commandes de nettoyage . . . 19

6 Conclusion générale **20**
6.1 Compétences acquises 20
6.2 Points d'attention pour la sécurité 20

1. Mini-TP 3 – Démarrage Docker & MongoDB

1.1 Objectif

L'objectif de ce premier mini-TP est de lancer un conteneur MongoDB via Docker et de vérifier son bon fonctionnement. C'est la base indispensable pour tous les travaux pratiques suivants.

1.2 Commandes exécutées

Étape 1 – Lancement du conteneur MongoDB

```
docker run -d --name mongo-noseclab -p 27018:27017 -v mongo-data:/data/db mongo:7
```

Explication :

- `-d` : mode détaché (le conteneur tourne en arrière-plan)
- `-name mongo-noseclab` : nom du conteneur
- `-p 27018:27017` : redirection du port 27018 de l'hôte vers le port 27017 du conteneur
- `-v mongo-data:/data/db` : volume Docker pour persister les données
- `mongo:7` : image officielle MongoDB version 7

Étape 2 – Vérification du conteneur

```
docker ps
```

Résultat obtenu :

```
CONTAINER ID   IMAGE     COMMAND                  STATUS
            PORTS                        NAMES
a1b2c3d4e5f6   mongo:7   "docker-entrypoint. s  "   Up 30
   seconds   0.0.0.0:27018->27017/tcp   mongo-noseclab
```

Étape 3 – Vérification des logs

```
docker logs mongo-noseclab | more
```

Résultat obtenu :

```
{"t":{"$date":"..."},"s":"I","c":"NETWORK","id":23016,"msg":"
   Waiting for connections","attr":{"port":27017,"ssl":"off
   "}}
```

Étape 4 – Connexion à mongosh

```
docker exec -it mongo-noseclab mongosh
```

Étape 5 – Test de connexion

```
db.runCommand({ ping: 1 })
```

Résultat obtenu :

```
{ ok: 1 }
```

1.3 Bilan du Mini-TP 3

- Le conteneur MongoDB est correctement lancé sur le port 27018
- La connexion via `mongosh` est fonctionnelle
- La commande `ping` confirme que le serveur répond
- Les données seront persistées grâce au volume `mongo-data`

2. Mini-TP 4 – Création de la base et insertion du dataset

2.1 Objectif

Créer la base de données `noseclab`, la collection `security_logs`, et insérer un jeu de données réaliste de 12 logs de sécurité.

2.2 Commandes exécutées

Création et utilisation de la base

```
use noseclab
```

Résultat obtenu :

```
switched to db noseclab
```

2.2.1 Insertion des 12 logs de sécurité

Insertion des documents

```
db.security_logs.insertMany([
  {
    timestamp: ISODate("2026-02-25T08:01:00Z"),
    src_ip: "192.168.1.10",
    user: "alice",
    action: "login",
    status: "success",
    device: "laptop",
    geo: { country: "FR", city: "Paris" },
    severity: "info"
  },
  {
    timestamp: ISODate("2026-02-25T08:05:23Z"),
    src_ip: "192.168.1.10",
    user: "alice",
    action: "file_access",
    status: "success",
    device: "laptop",
    geo: { country: "FR", city: "Paris" },
    severity: "info"
  },
  {
    timestamp: ISODate("2026-02-25T08:12:45Z"),
    src_ip: "10.0.0.50",
    user: "bob",
    action: "login",
    status: "failure",
    device: "desktop",
    geo: { country: "FR", city: "Lyon" },
    severity: "warning"
  },
  {
    timestamp: ISODate("2026-02-25T08:13:10Z"),
    src_ip: "10.0.0.50",
    user: "bob",
    action: "login",
    status: "failure",
    device: "desktop",
    geo: { country: "FR", city: "Lyon" },
    severity: "warning"
  },
  {
    timestamp: ISODate("2026-02-25T08:13:42Z"),
    src_ip: "10.0.0.50",
    user: "bob",
    action: "login",
```

Résultat obtenu :

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('...'),
    '1': ObjectId('...'),
    ...
    '11': ObjectId('...')
  }
}
```

2.3 Vérifications

Lister les bases

```
show dbs
```

Résultat obtenu :

```
admin     40 kB
config   12 kB
local     40 kB
noseclab  40 kB
```

Lister les collections

```
show collections
```

Résultat obtenu :

```
security_logs
```

Compter les documents

```
db.security_logs.countDocuments()
```

Résultat obtenu :

```
12
```

Aperçu d'un document

```
db.security_logs.findOne()
```

Résultat obtenu :

```
{
  _id: ObjectId('...'),
  timestamp: ISODate('2026-02-25T08:01:00Z'),
  src_ip: '192.168.1.10',
  user: 'alice',
  action: 'login',
  status: 'success',
  device: 'laptop',
  geo: { country: 'FR', city: 'Paris' },
  severity: 'info'
}
```

2.4 Bilan du Mini-TP 4

- La base `noseclab` est créée
- La collection `security_logs` contient 12 documents
- Les logs couvrent plusieurs types d'événements : login, file_access, port_scan, logout, privilege_escalation
- Plusieurs pays sont représentés : France (FR), Chine (CN), États-Unis (US)
- Plusieurs sévérités : info, warning, high, critical

3. Mini-TP 5 – Exercices CRUD guidés

3.1 Objectif

Maîtriser les quatre opérations fondamentales CRUD (Create, Read, Update, Delete) sur la collection `security_logs`.

3.2 Exercice A – Create (Ajouter un document)

Insertion d'un nouveau log

```
db.security_logs.insertOne({
  timestamp: ISODate("2026-02-25T11:45:00Z"),
  src_ip: "198.51.100.7",
  user: "eve",
  action: "login",
  status: "failure",
  device: "smartphone",
  geo: { country: "US", city: "New York" },
  severity: "warning"
})
```

Résultat obtenu :

```
{ acknowledged: true, insertedId: ObjectId('...') }
```

Vérification du nombre de documents

```
db.security_logs.countDocuments()
```

Résultat obtenu :

```
13
```

3.3 Exercice B – Read (Lire et filtrer)

B1 – Tous les logs avec statut failure

```
db.security_logs.find({ status: "failure" })
```

Explication :

Retourne environ 6 documents : les 3 échecs de bob, les 2 tentatives depuis la Chine, et l'échec d'eve.

B2 – Tous les logs de l'utilisateur bob

```
db.security_logs.find({ user: "bob" })
```

Résultat obtenu :

4 documents (3 failures + 1 success)

B3 – Logs de sévérité critical

```
db.security_logs.find({ severity: "critical" })
```

Résultat obtenu :

2 documents (port_scan + privilege_escalation)

B4 – Logs provenant de Chine

```
db.security_logs.find({ "geo.country": "CN" })
```

Résultat obtenu :

2 documents (les tentatives depuis Beijing)

Explication :

La notation pointée `"geo.country"` permet d'accéder aux champs imbriqués dans l'objet `geo`.

3.4 Exercice C – Update (Modifier des documents)

C1 – Modification du port_scan (faux positif)

```
db.security_logs.updateOne(
  { src_ip: "172.16.0.99", action: "port_scan" },
  { $set: { investigated: true, severity: "info" } }
)
```

Résultat obtenu :

```
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
```

Vérification

```
db.security_logs.findOne({ src_ip: "172.16.0.99" })
```

Résultat obtenu :

Le document contient maintenant les champs `investigated: true` et `severity: "info"`.

C2 – Augmentation de la sévérité de tous les login failure

```
db.security_logs.updateMany(
  { action: "login", status: "failure" },
  { $set: { severity: "high" } }
)
```

Résultat obtenu :

```
{ acknowledged: true, matchedCount: 6, modifiedCount: 6 }
```

3.5 Exercice D – Delete (Supprimer un document)

Suppression du log d'eve

```
db.security_logs.deleteOne({ user: "eve", src_ip:
   "198.51.100.7" })
```

Résultat obtenu :

```
{ acknowledged: true, deletedCount: 1 }
```

Vérification finale

```
db.security_logs.countDocuments()
```

Résultat obtenu :

```
12
```

3.6 Commandes dangereuses (à ne JAMAIS exécuter en production)

DANGER – Suppression de tous les documents

```
// db.security_logs.deleteMany({})
// db.security_logs.drop()
```

Explication :

Ces commandes supprimeraient définitivement toutes les preuves d'intrusion. En contexte SOC, cela équivaudrait à effacer des preuves forensiques critiques.

3.7 Bilan du Mini-TP 5

Opération	**Commande MongoDB**	**Exemple**
Create	insertOne() / insertMany()	Ajout du log d'eve
Read	find() / findOne()	Filtrage par status, user, severity
Update	updateOne() / updateMany()	Modification du port_scan
Delete	deleteOne() / deleteMany()	Suppression du log d'eve

4. Mini-TP 6 – Scénarios de requêtes cybersécurité

4.1 Objectif

Utiliser les requêtes MongoDB pour répondre à des questions d'investigation sécurité typiques d'un analyste SOC.

4.2 Scénario 1 – Détection de brute-force

Tentatives de login échouées de bob

```
db.security_logs.find({
  user: "bob",
  action: "login",
  status: "failure"
})
```

Résultat obtenu :

```
[
  { timestamp: ISODate("2026-02-25T08:12:45Z"), src_ip:
     "10.0.0.50", ... },
  { timestamp: ISODate("2026-02-25T08:13:10Z"), src_ip:
     "10.0.0.50", ... },
  { timestamp: ISODate("2026-02-25T08:13:42Z"), src_ip:
     "10.0.0.50", ... }
]
```

Explication :

Analyse SOC : 3 échecs rapprochés depuis la même IP (10.0.0.50) dans un intervalle d'environ 1 minute. C'est un indicateur fort d'attaque par brute-force. L'analyste doit déclencher une alerte et investiguer.

4.3 Scénario 2 – Activité depuis l'étranger

Logs provenant de l'étranger (hors France)

```
db.security_logs.find({
  "geo.country": { $ne: "FR" }
})
```

Résultat obtenu :

Retourne les 2 tentatives depuis la Chine (CN) mais aussi les documents sans champ `geo` (comme le port_scan).

Version corrigée (exclut les documents sans geo)

```
db.security_logs.find({
  "geo.country": { $exists: true, $ne: "FR" }
})
```

Explication :

Piège identifié : L'opérateur $ne retourne `true` pour les champs inexistants (undefined "FR"). Pour exclure ces cas, il faut ajouter `$exists: true`.

4.4 Scénario 3 – Les 3 derniers événements critiques

Derniers événements high/critical avec projection

```
db.security_logs.find(
  { severity: { $in: ["high", "critical"] } },
  { timestamp: 1, src_ip: 1, action: 1, severity: 1, _id: 0 }
).sort({ timestamp: -1 }).limit(3)
```

Explication :

- `$in : ["high", "critical"]` : filtre sur les sévérités
- Projection : ne retourne que les champs nécessaires (pas _id)
- `sort({timestamp: -1})` : tri décroissant (du plus récent au plus ancien)
- `limit(3)` : seulement les 3 premiers

4.5 Scénario 4 – Nombre d'événements par type d'action (Aggregation)

Statistiques par type d'action

```
db.security_logs.aggregate([
  { $group: { _id: "$action", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])
```

Résultat obtenu :

```
[
  { _id: "login", count: 8 },
  { _id: "logout", count: 1 },
  { _id: "file_access", count: 1 },
  { _id: "port_scan", count: 1 },
  { _id: "privilege_escalation", count: 1 }
]
```

Explication :

L'aggregation pipeline est introduite ici. Elle sera approfondie en Séance 2. Les étapes sont :

1. $group : regroupe les documents par la valeur du champ `action`
2. $sum : compte le nombre de documents dans chaque groupe
3. $sort : trie par ordre décroissant du compteur

4.6 Scénario 5 – Exercice libre

Appareils utilisés pour des connexions réussies

```
db.security_logs.find(
  { action: "login", status: "success" },
  { user: 1, device: 1, timestamp: 1, _id: 0 }
).sort({ timestamp: 1 })
```

Résultat obtenu :

```
[
  { user: "alice", device: "laptop", timestamp: ISODate
     ("2026-02-25T08:01:00Z") },
  { user: "bob", device: "desktop", timestamp: ISODate
     ("2026-02-25T08:15:00Z") },
  { user: "charlie", device: "server-web01", timestamp:
     ISODate("2026-02-25T11:00:00Z") }
]
```

Explication :

Cette requête répond à la question : "Quels appareils ont été utilisés pour des connexions réussies ?" en affichant uniquement les champs demandés, triés du plus ancien au plus récent.

4.7 Bilan du Mini-TP 6

- Maîtrise des filtres complexes ($ne, $in, $exists)
- Utilisation de la projection pour limiter les données retournées
- Découverte de l'aggregation pipeline avec $group
- Identification d'un piège classique : les champs inexistants avec $ne
- Application concrète à des cas d'usage SOC

5. Synthèse et commandes de nettoyage

5.1 Récapitulatif des commandes essentielles

Commande	**Description**
docker ps	Voir les conteneurs actifs
docker logs mongo-noseclab	Voir les logs MongoDB
docker exec -it mongo-noseclab mongosh	Entrer dans mongosh
use noseclab	Changer de base de données
show collections	Lister les collections
db.collection.find()	Rechercher des documents
db.collection.findOne()	Afficher un document
db.collection.countDocuments()	Compter les documents
db.collection.insertOne()	Insérer un document
db.collection.updateOne()	Modifier un document
db.collection.deleteOne()	Supprimer un document
db.collection.aggregate()	Pipeline d'aggregation
quit()	Quitter mongosh

5.2 Commandes de nettoyage

Nettoyage complet de l'environnement

```
# Quitter mongosh
quit()

# Arr ter le conteneur
docker stop mongo-noseclab

# Supprimer le conteneur
docker rm mongo-noseclab

# Supprimer le volume (optionnel - supprime toutes les
   donnees)
docker volume rm mongo-data
```

6. Conclusion générale

6.1 Compétences acquises

À l'issue de ces travaux pratiques, j'ai acquis les compétences suivantes :

1. **Déploiement Docker** : Lancement d'un conteneur MongoDB avec persistance des données via volumes Docker.
2. **Création de base NoSQL** : Création d'une base `noseclab` et d'une collection `security_logs` sans schéma prédéfini.
3. **Insertion de données** : Utilisation de `insertMany()` pour insérer un dataset réaliste de logs de sécurité.
4. **Opérations CRUD** : Maîtrise complète de Create, Read, Update, Delete sur des documents MongoDB.
5. **Requêtes avancées** : Filtrage avec conditions ($ne, $in, $exists), projection, tri, limitation.
6. **Aggregation** : Première approche du pipeline $group pour les statistiques.
7. **Contexte SOC** : Application des requêtes à des cas concrets (détection brute-force, activités étrangères, événements critiques).

6.2 Points d'attention pour la sécurité

— **Jamais en production** : `deleteMany({})` ou `drop()` supprimeraient définitivement les données.

— **Projection systématique** : Ne retourner que les champs nécessaires pour éviter l'exposition de données sensibles.

— **Attention aux champs inexistants** : L'opérateur $ne retourne `true` pour les champs absents.

— **Validation des entrées** : MongoDB est schema-less, l'application doit valider les données avant insertion.